

Resilient and Secure Systems through Verified Security Policies (RSVP)

Graeme Smith, Kirsten Winter, Kostyantyn Vorobyov
Davin McCall, Rebecca O'Donoghue, Gabriel Nunes

Motivation

Policies-as-code

- All accesses to information are controlled by security policies
- Automatically enforced at run-time
- Manually checking and maintaining policies is infeasible on large-scale systems

RSVP

- A user-oriented tool to develop and adapt security policies over time, while maintaining high assurance of overall security

RSVP

Support during policy development and analysis

- Detect redundant, duplicate, or irrelevant policies
- Understand policy effects
- Analyse change impact of policy changes
- Add assurance during policy development, refactoring or debugging
- Reinforce security boundaries of a system
- Detect conflicting policies (interoperability in different environments)
- Policy Coverage

Redundant and Irrelevant Policies

Redundancy

- One policy's effects are a superset of another (*subsumption*)
- One policy has same effects as another (*duplication*)

Irrelevance

- A policy has no effect on what is allowed e.g., impossible condition

Contradiction

- Effects of one policy are negated by another

Change Impact

Policies may be changed rapidly

- Policies can be added, removed, or refactored
- Policies can be patched automatically

RSVP can quantify and compare policy accesses

- The effect of such changes can be analysed

Security Boundaries

Policies can evolve over time even if security requirements of the system remain the same

Need to ensure that security policy configuration provides the same level of protection even if policies change

RSVP can formally express and enforce security boundaries of a system ensuring the same level of protection on policy updates

RSVP Prototype

Built using relational reasoning

- Policies modelled as relations
- Computation delegated to Souffle Datalog
- Closed-world reasoning

Web-based UI

Uses Cedar authorisation language

- Built to support multiple authorisation frameworks

Principal is allowed to perform ***Action*** on ***Resource***

(User::"Alice", Action::"ViewPhoto", Photo::"Lake.jpg")

(User::"Bob", Action::"UploadPhoto", Album::"Vacation")

Cedar Schema

```
entity Clinic;
```

```
entity Role enum ["Administrator", "Assistant", "Doctor"];
```

```
entity Employee = {  
    name: String,  
    clinics: Set<Clinic>,  
    roles: Set<Role>  
};
```

```
action ListEmployees appliesTo {  
    principal: Employee,  
    resource: Clinic  
};
```

```
permit(  
    principal is Employee,  
    action == Action::"ListEmployees",  
    resource is Clinic)  
when {  
    principal.clinics.contains(resource)  
};
```

Cedar Entities

```
[
  {
    "uid": { "type": "Clinic", "id": "North" }
  },
  {
    "uid": { "type": "Role", "id": "Doctor" }
  },
  {
    "uid": { "type": "Employee", "id": "James Carter" },
    "attrs": {
      "name": "James Carter",
      "clinics": [
        { "type": "Clinic", "id": "A" }
      ],
      "roles": [
        { "type": "Role", "id": "Doctor" }
      ]
    }
  }
]
```

Demo

Moving Forward

- Information Flow Analysis
- Supporting other policy languages: Zanzibar, Open FGA, Rego
- Unifying policy specifications for full system security posture analysis
- Open-world policy reasoning and analysis
- From prototype to industry-ready tool
 - Database integration
 - Scaling to realistic policy sets